

Subset sum queries using bitset

Last Updated : 23 Jul, 2025



Given an array `arr[]` and a number of queries, where in each query we have to check whether a subset whose sum is equal to given number exists in the array or not.

Examples:

```
Input : arr[] = {1, 2, 3};
        query[] = {5, 3, 8}
Output : Yes, Yes, No
There is a subset with sum 5, subset is {2, 3}
There is a subset with sum 3, subset is {1, 2}
There is no subset with sum 8.
Input : arr[] = {4, 1, 5};
        query[] = {7, 9}
Output : No, Yes
There is no subset with sum 7.
There is a subset with sum 9, subset is {4, 5}
```

The idea is to use [bitset container in C++](#). Using bitset, we can **precalculate the existence all the subset sums in an array** in $O(n)$ and answer subsequent queries in just $O(1)$. We basically use an array of bits `bit[]` to represent the subset sum of elements in the array. Size of `bit[]` should be at least sum of all array elements plus 1 to answer all queries. We keep `bit[x]` as 1 if `x` is a subset sum of given array, else false. Note that indexing is assumed to begin with 0.

```
For every element arr[i] of input array,
we do following
// bit[x] will be 1 if x is a subset
// sum of arr[], else 0
bit = bit | (bit << arr[i])
```

How does this work?

Let us consider `arr[] = {3, 1, 5}`, we need to check whether a subset sum of `x` exists or not, where $0 \leq x \leq \sum arr_i$.

We create a bitset `bit[10]` and reset all the bits to 0, i.e., we make it `0000000000`.

Set the 0th bit, because a subset sum of 0 exists in every array.

Now, the bit array is `0000000001`

Apply the above technique for all the elements of the array :

Current bitset = `0000000001`

After doing `"bit = bit | (bit << 3)"`,

bitset becomes `0000001001`

After doing `"bit | (bit << 1)"`,

bitset becomes `0000011011`

After doing `"bit | (bit << 5)"`,

bitset becomes `1101111011`

Finally, we have the bit array as `1101111011`, so, if `bit[x]` is 1 then a subset sum of `x` exists otherwise not. We can clearly observe that a subset sum of all the numbers from 0 to 9 except 2 and 7 exists in the array.

Implementation:

C++ Java C# JavaScript Python3

```
// C++ program to answer subset sum queries using bitset
#include <bits/stdc++.h>
using namespace std;

// Maximum allowed query value
# define MAXSUM 10000

// function to check whether a subset sum equal to n
// exists in the array or not.
void processQueries(int query[], int nq, bitset<MAXSUM> bit)
{
    // One by one process subset sum queries
    for (int i=0; i<nq; i++)
    {
```

```

int x = query[i];

// If x is beyond size of bit[]
if (x >= MAXSUM)
{
    cout << "NA, ";
    continue;
}

// Else if x is a subset sum, then x'th bit
// must be set
bit[x]? cout << "Yes, " : cout << "No, ";
}
}

// function to store all the subset sums in bit vector
void preprocess(bitset<MAXSUM> &bit, int arr[], int n)
{
    // set all the bits to 0
    bit.reset();

    // set the 0th bit because subset sum of 0 exists
    bit[0] = 1;

    // Process all array elements one by one
    for (int i = 0; i < n; ++i)

        // Do OR of following two
        // 1) All previous sums. We keep previous value
        // of bit.
        // 2) arr[i] added to every previous sum. We
        // move all previous indexes arr[i] ahead.
        bit |= (bit << arr[i]);
}

// Driver program
int main()
{
    int arr[] = {3, 1, 5};
    int query[] = {8, 7};

    int n = sizeof(arr) / sizeof(arr[0]);

```

```
int nq = sizeof(query) / sizeof(query[0]);

// a vector of MAXSUM number of bits
bitset<MAXSUM> bit;

preprocess(bit, arr, n);
processQueries(query, nq, bit);

return 0;
}
```

Output

Yes, No,

Time complexity : $O(n * \text{MAX_ELEMENT})$ for pre-calculating since **left shift** operator takes $O(q)$ for $p < q$. It takes $O(1)$ for subsequent queries, where n is the number of elements in the array.

Auxiliary Space: $O(n)$

If you like GeeksforGeeks and would like to contribute, you can also write an article using [write.geeksforgeeks.org](https://www.geeksforgeeks.org/write-a-contribution/) or mail your article to review-team@geeksforgeeks.org. See your article appearing on the GeeksforGeeks main page and help other Geeks.

 Comment

More info 

Advertise with us

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information